LAPORAN TUGAS BESAR
KRIPTOGRAFI

Semester I 2026/2027

TUNG-128



Disusun oleh:

Kelompok 16

Orvin Andika Ikhsan Abhista	13523017
Fajar Kurniawan	13523027
Rafizan Muhammad Syawalazmi	13523034
Aryo Wisanggeni	13523100

Program Studi Teknik informatika

Institut Teknologi Bandung

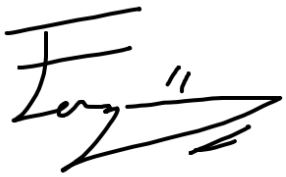
2026

Daftar Isi

LAPORAN TUGAS BESAR **1**
Daftar Isi **2**
Pernyataan Tidak Melakukan Kecurangan **3**
Pendahuluan **4**
I. Dasar Teori **5**
1.1 Konsep Dasar Cipher Blok 5
1.2 Substitusi dan Permutasi 5
1.2.1 Substitusi dan Kotak-S 5
1.2.2 Permutasi 7
1.3 Confusion dan Diffusion 7
1.4 Cipher Berulang dan Kunci Putaran 7
1.5 Jaringan Feistel 8
1.6 Mode Operasi 9
1.7 Operasi Modular 10
1.8 Metrik Analisis Keamanan 11
II. Perancangan Block Cipher **12**
2.1 Gambaran Umum TUNG-128 12
2.2 Proses Enkripsi 13
2.3 Proses Dekripsi 14
2.4 Komponen Algoritma 15
2.4.1 Fungsi Putaran 15
2.4.2 Kotak-S 16
2.4.3 Permutasi Byte 18
2.4.4 Perkalian Modular 19
2.4.5 Rotasi Bit 20
2.4.6 Pembangkitan Kunci Putaran 20
III. Implementasi dan Simulasi **23**
3.1. Deskripsi Program 23
3.2. Deskripsi Antarmuka 23
3.3. Hasil Pengujian Enkripsi dan Dekripsi 24
IV. Analisis Keamanan **28**
V. Kesimpulan dan Saran Pengembangan **31**
5.1. Kesimpulan 31
5.2. Saran Pengembangan 31
VI. Daftar Pustaka **32**
Lampiran **33**

Pernyataan Tidak Melakukan Kecurangan

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

 Orvin Andhika Ikhsan Abhista 13523017	 Fajar Kurniawan 13523027
 Rafizan Muhammad Syawalazmi 13523034	 Aryo Wisanggeni 13523100

Pendahuluan

Puji syukur kami panjatkan ke Tuhan Yang Maha Esa karena atas rahmat-Nya kami dapat menyelesaikan Tugas 2 IF4020 Kriptografi, yaitu perancangan dan implementasi sebuah algoritma *block cipher* baru beserta laporannya.

Kami mengucapkan terima kasih kepada Prof. Dr. Ir. Rinaldi, M.T. selaku dosen mata kuliah IF4020 Kriptografi yang telah memberikan materi mengenai *block cipher*. Materi tersebut dimulai dari definisi *block cipher*, mode operasi, DES, AES, hingga berbagai variasi *block cipher* lainnya. Materi yang diberikan tersebut secara langsung membantu kami dalam merancang algoritma dan menyelesaikan tugas ini.

Kami juga mengucapkan terima kasih kepada asisten mata kuliah, Ghana dan Evander, yang telah menyusun spesifikasi tugas ini. Terima kasih pula atas pengunduran tenggat pengumpulan selama tiga hari, sehingga kami dan seluruh peserta kuliah Kriptografi memiliki waktu yang cukup untuk mempersiapkan UTS sekaligus menyelesaikan tugas ini.

Kami menyadari bahwa laporan dan algoritma yang kami rancang masih memiliki ruang untuk berkembang. Oleh karena itu, kami terbuka terhadap kritik dan saran yang membangun. Semoga laporan ini bermanfaat bagi pembaca.

I. Dasar Teori

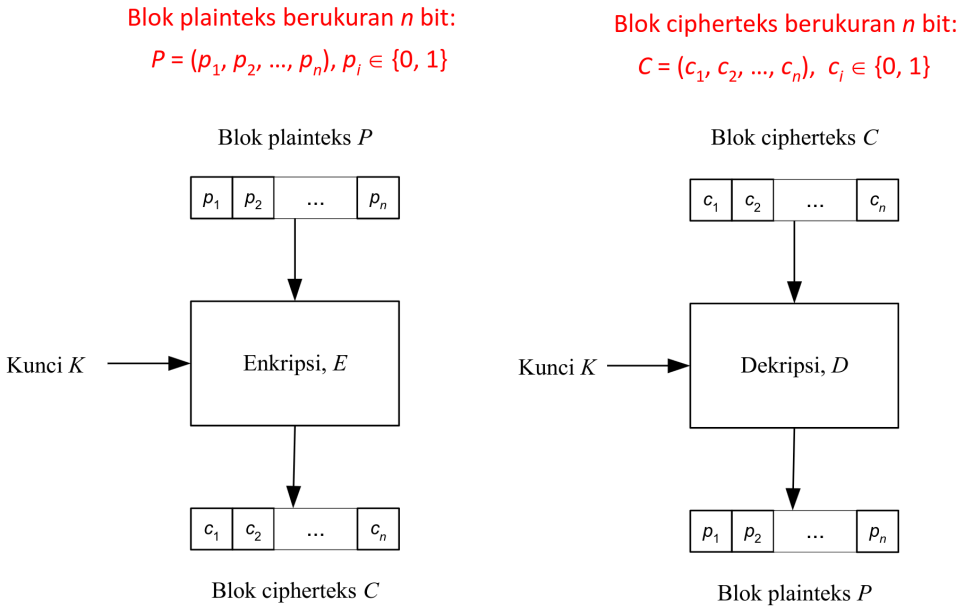
1.1 Konsep Dasar Cipher Blok

Cipher blok (*block cipher*) adalah algoritma kriptografi simetri yang membagi plainteks menjadi blok-blok bit berukuran tertentu, lalu mengenkripsi setiap blok menjadi blok cipherteks berukuran sama dengan menggunakan kunci yang sama. Jika ukuran blok adalah n bit, blok plainteks dinyatakan sebagai $P = (p_1, p_2, \ldots, p_n)$ dan blok cipherteks sebagai $C = (c_1, c_2, \ldots, c_n)$, dengan $p_i, c_i \in \{0, 1\}$.

Enkripsi dan dekripsi dengan kunci K dinyatakan sebagai:

$$C = E_K(P), \qquad P = D_K(C)$$

Fungsi D adalah invers dari fungsi E, sehingga $D_K(E_K(P)) = P$.



Gambar 1.1 Skema enkripsi dan dekripsi pada cipher blok (Munir, 2026)

Dua parameter utama cipher blok adalah ukuran blok dan ukuran kunci. Ukuran blok selalu tetap selama enkripsi dan dekripsi; makin besar ukuran blok, makin besar efek *diffusion* yang dapat dicapai. Ukuran kunci dapat sama dengan atau berbeda dari ukuran blok; makin besar ukuran kunci, makin besar efek *confusion* dan makin tahan cipher terhadap serangan *brute force*.

1.2 Substitusi dan Permutasi

Cipher blok modern tetap dibangun dari dua teknik dasar kriptografi klasik, yaitu substitusi dan permutasi (transposisi), tetapi keduanya diterapkan pada bit atau byte.

1.2.1 Substitusi dan Kotak-S

Substitusi mengganti sekelompok bit dengan sekelompok bit lain. Pada cipher blok, substitusi umumnya direalisasikan dengan tabel yang disebut kotak-S (*S-box*) dengan nilai masukan dipakai sebagai indeks, dan isi tabel pada indeks tersebut menjadi keluarannya (*table lookup*). Agar dekripsi dapat dilakukan, kotak-S harus bijektif sehingga memiliki kotak-S invers.

Isi kotak-S tidak harus dipilih secara acak. Pada AES Rijndael, kotak-S dibangkitkan lewat perhitungan dengan dua langkah:

1. Setiap byte diganti dengan invers perkaliannya di dalam GF(2^8); nilai 00 dipetakan ke dirinya sendiri.
2. Hasilnya dikenai transformasi *affine* pada tingkat bit, yaitu perkalian dengan sebuah matriks bit lalu penjumlahan (XOR) dengan sebuah konstanta.

Langkah pertama memberikan sifat nirlanjar (non-linear) yang kuat, sedangkan langkah kedua menghilangkan struktur aljabar sederhana dari operasi invers dan mencegah adanya titik tetap.

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Gambar 1.2 Transformasi affine pada pembangkitan kotak-S AES (Munir, 2026)

Galois Field GF(2^8). GF(2^m) adalah medan berhingga biner yang elemennya berupa string biner sepanjang m bit. Setiap elemen dapat dinyatakan sebagai polinom berderajat paling tinggi $m - 1$ dengan koefisien 0 atau 1. Sebagai contoh, byte 01010111 di dalam GF(2^8) adalah polinom $x^6 + x^4 + x^2 + x + 1$. Operasi aritmetikanya adalah sebagai berikut:

- **Penjumlahan:** koefisien yang bersesuaian dijumlahkan dalam modulo 2, sehingga penjumlahan sama dengan operasi XOR.
- **Perkalian:** kedua polinom dikalikan, lalu hasilnya dibagi dengan sebuah polinom tak-tereduksi (*irreducible polynomial*) berderajat m; sisa pembagiannya adalah hasil perkalian. AES memakai polinom $x^8 + x^4 + x^3 + x + 1$.
- **Invers perkalian:** setiap elemen tak-nol a memiliki invers a^{-1} sehingga $a \cdot a^{-1} = 1$. Karena $a^{255} = 1$ untuk setiap a tak-nol di GF(2^8), invers dapat dihitung sebagai $a^{-1} = a^{254}$.

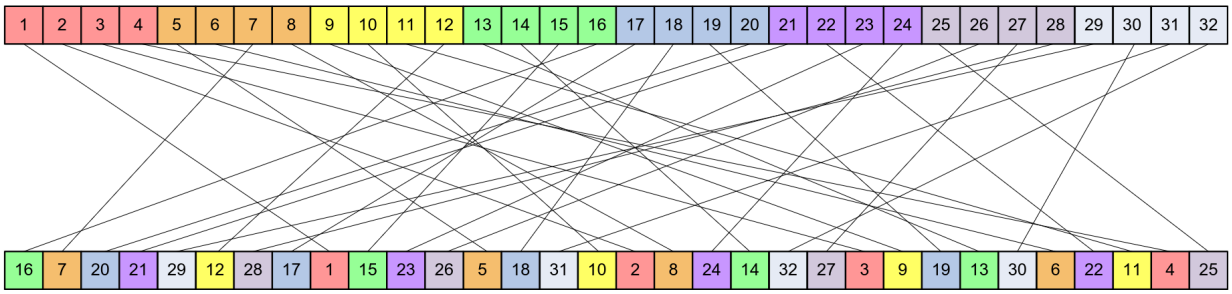
Pemilihan polinom tak-tereduksi yang berbeda menghasilkan medan dengan tabel perkalian yang berbeda, sehingga kotak-S yang dihasilkan juga berbeda.

1.2.2 Permutasi

Permutasi mengacak susunan bit atau byte di dalam blok tanpa mengubah nilainya. Terdapat dua cara umum untuk merealisasikannya:

- **Kotak-P (*P-box*):** sebuah tabel yang setiap entrinya menyatakan posisi asal bit atau byte yang menempati posisi tersebut. Contohnya adalah permutasi awal pada DES.
- **Pergeseran sirkuler (rotasi):** bit atau byte digeser ke kiri atau ke kanan sejauh *k* posisi, dan yang keluar dari satu ujung masuk kembali dari ujung lainnya. Contohnya adalah ShiftRows pada AES.

Permutasi bersifat bijektif, sehingga inversnya diperoleh dengan membalik pemetaan posisi.



Gambar 2.3 Permutasi bit di dalam sebuah blok (Munir, 2026)

1.3 Confusion dan Diffusion

Claude Shannon, dalam makalahnya *Communication Theory of Secrecy Systems* (1949), memperkenalkan dua prinsip perancangan cipher yang aman, yaitu *confusion* dan *diffusion*.

- **Confusion** menyembunyikan hubungan statistik antara plainteks, cipherteks, dan kunci, sehingga hubungan antara cipherteks dan kunci menjadi serumit mungkin. Prinsip ini dicapai dengan operasi substitusi, yaitu kotak-S.
- **Diffusion** menyebarkan pengaruh satu bit plainteks atau kunci ke sebanyak mungkin bit cipherteks, sehingga pola statistik plainteks tidak tampak pada cipherteks. Prinsip ini dicapai dengan operasi permutasi.

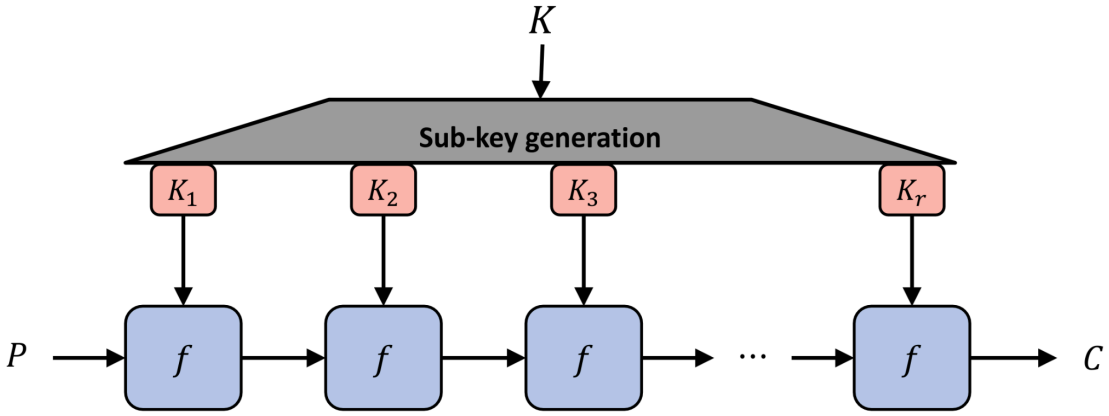
Satu kali penerapan substitusi dan permutasi belum cukup. Karena itu kedua operasi diulang berkali-kali di dalam satu cipher agar confusion dan diffusion menumpuk.

1.4 Cipher Berulang dan Kunci Putaran

Cipher berulang (*iterated cipher*) melakukan proses *enciphering* yang sama sejumlah putaran (*round*) untuk menghasilkan cipher yang lebih kuat. Cipher berulang dinyatakan sebagai:

$$C_i = f(C_{i-1}, K_i), \qquad i = 1, 2, \ldots, r$$

Di sini r adalah jumlah putaran, K_i adalah kunci putaran (*round key*) ke-i, dan f adalah fungsi putaran (*round function*) yang berisi operasi substitusi, permutasi, dan operasi lain seperti ekspansi atau kompresi. Plainteks dinyatakan sebagai C_0 dan cipherteks sebagai C_r.



Gambar 1.4 Struktur cipher berulang dengan *r* putaran (Munir, 2026)

Pada pembengkitan kunci putaran, semakin banyak putaran, semakin besar efek diffusion, tetapi komputasi semakin lambat. Jumlah putaran dipilih merupakan keseimbangan antara keamanan dan efisiensi.

Setiap putaran memakai kunci putaran yang berbeda. Kunci putaran dibangkitkan dari kunci eksternal (*master key*) milik pengguna melalui proses yang disebut *key expansion* atau *key schedule*. Proses ini sengaja dibuat kompleks agar kunci putaran berbeda satu sama lain dan sulit dihubungkan kembali ke kunci eksternal. Panjang kunci putaran tidak harus sama dengan panjang kunci eksternal.

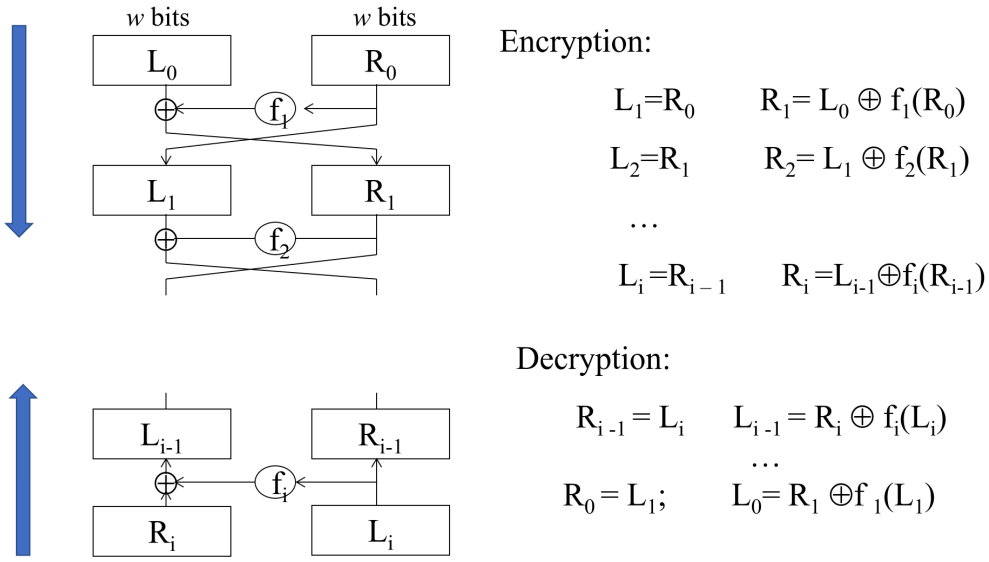
1.5 Jaringan Feistel

Jaringan Feistel adalah struktur cipher berulang yang dipakai antara lain oleh DES, GOST, RC5, dan RC6. Blok berukuran 2w bit dibagi menjadi dua bagian sama besar, yaitu bagian kiri L dan bagian kanan R. Pada putaran ke-i berlaku:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

Dekripsi dilakukan dengan membalik urutan komputasi dan memakai kunci putaran dalam urutan terbalik:

$$R_{i-1} = L_i, \qquad L_{i-1} = R_i \oplus f(L_i, K_i)$$



Gambar 1.5 Jaringan Feistel (Munir, 2026)

Jaringan Feistel bersifat reversibel karena sifat operasi XOR:

$$L_{i-1} \oplus f(R_{i-1}, K_i) \oplus f(R_{i-1}, K_i) = L_{i-1}$$

Sifat ini memiliki dua akibat penting:

- Algoritma dekripsi tidak perlu dirancang terpisah; struktur yang sama dipakai untuk enkripsi dan dekripsi.
- Sifat reversibel tidak bergantung pada fungsi *f*, sehingga *f* tidak harus memiliki invers dan dapat dibuat serumit mungkin.

1.6 Mode Operasi

Mode operasi menentukan cara blok-blok pesan diproses ketika panjang pesan melebihi satu blok. Tabel 2.1 merangkum lima mode standar, dengan E_K dan D_K sebagai enkripsi dan dekripsi satu blok.

Tabel 1.1 Rumus enkripsi dan dekripsi lima mode operasi

Mode	Enkripsi	Dekripsi	Nilai awal
ECB	$C_i = E_K(P_i)$	$P_i = D_K(C_i)$	–
CBC	$C_i = E_K(P_i \oplus C_{i-1})$	$P_i = D_K(C_i) \oplus C_{i-1}$	C_0 = IV
CFB	$C_i = P_i \oplus E_K(C_{i-1})$	$P_i = C_i \oplus E_K(C_{i-1})$	C_0 = IV
OFB	$O_i = E_K(O_{i-1})$, $C_i = P_i \oplus O_i$	$O_i = E_K(O_{i-1})$, $P_i = C_i \oplus O_i$	O_0 = IV
CTR	$C_i = P_i \oplus E_K(T_i)$	$P_i = C_i \oplus E_K(T_i)$	T_1 = counter awal,

			$T_{i+1} = T_i + 1$

Rumus CFB dan OFB di atas adalah untuk kasus ukuran umpan balik sama dengan ukuran blok.

- **Electronic Code Book (ECB).** Setiap blok dienkripsi secara individual dan independen. Mode ini sederhana dan dapat diparalelkan, tetapi blok plainteks yang sama selalu menghasilkan blok cipherteks yang sama, sehingga pola pada plainteks tetap terlihat dan cipherteks mudah dimanipulasi.
- **Cipher Block Chaining (CBC).** Setiap blok plainteks di-XOR-kan dengan blok cipherteks sebelumnya sebelum dienkripsi, sehingga setiap blok cipherteks bergantung pada semua blok sebelumnya. Blok pertama memakai *initialization vector* (IV). Blok plainteks yang sama tidak lagi menghasilkan blok cipherteks yang sama.
- **Cipher Feedback (CFB).** Blok cipherteks sebelumnya dienkripsi untuk menghasilkan kunci-alir (*keystream*), lalu kunci-alir di-XOR-kan dengan plainteks. Mode ini mengoperasikan cipher blok seperti cipher alir.
- **Output Feedback (OFB).** Mirip dengan CFB, tetapi yang diumpankan kembali adalah keluaran fungsi enkripsi, bukan cipherteks. Akibatnya kunci-alir tidak bergantung pada plainteks, dan kesalahan satu bit pada cipherteks hanya memengaruhi bit yang bersesuaian pada plainteks.
- **Counter (CTR).** Sebuah nilai *counter* dienkripsi untuk menghasilkan kunci-alir, lalu nilainya bertambah satu untuk blok berikutnya. Tidak ada perantaian antarblok, sehingga blok dapat diproses secara paralel.

1.7 Operasi Modular

Selain substitusi dan permutasi, banyak cipher blok memakai operasi aritmetika pada *word* berukuran w bit untuk menambah kompleksitas. Operasi yang umum adalah:

- **Penjumlahan modulo 2^w:** $a + b$ dihitung sebagai bilangan bulat, lalu hanya w bit terendah yang diambil. Operasi ini dipakai pada GOST dan RC5.
- **Perkalian modulo 2^w:** $a \times b$ dihitung sebagai bilangan bulat, lalu hanya w bit terendah yang diambil.
- **Pergeseran sirkuler:** $a <<< k$ menggeser a ke kiri sejauh k bit secara sirkuler.

Penjumlahan dan perkalian modulo 2^w bersifat nirlanjar (non-linear) terhadap operasi XOR karena adanya *carry* antarbit. Menggabungkan operasi dari kelompok aljabar yang berbeda (XOR, aritmetika modulo 2^w, dan rotasi) membuat cipher sulit dimodelkan dengan satu sistem persamaan sederhana.

Contoh penting adalah fungsi berikut pada RC6:

$$f(x) = x(2x + 1) \bmod 2^w$$

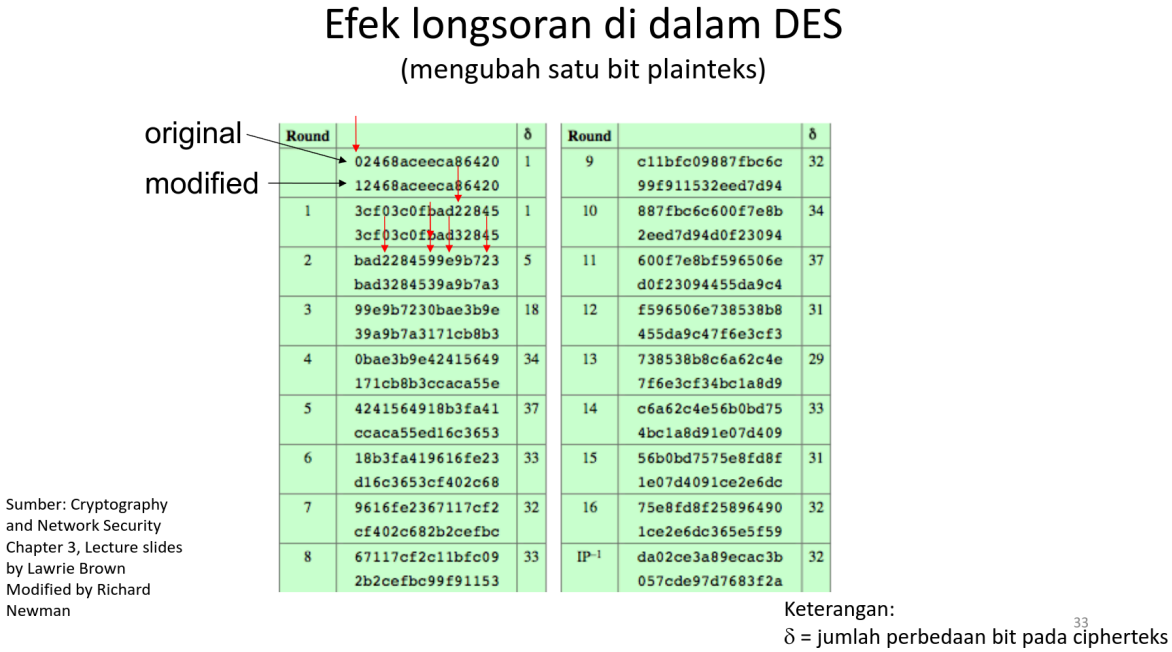
Pada fungsi ini setiap bit keluaran bergantung pada banyak bit masukan, terutama pada bit-bit orde tinggi. Karena itu RC6 selalu mengikutinya dengan pergeseran sirkuler agar pengaruh tersebut tersebar ke seluruh posisi bit.

1.8 Metrik Analisis Keamanan

Tiga metrik berikut umum dipakai untuk menilai kualitas cipherteks secara statistik.

Efek longsoran (*avalanche effect*). Efek longsoran adalah sifat ketika perubahan satu bit pada plainteks atau kunci menghasilkan perubahan besar pada cipherteks. Efek ini diukur sebagai persentase bit cipherteks yang berubah. Nilai idealnya mendekati 50%, yang berarti setiap bit cipherteks berubah dengan peluang setengah. Berikut adalah formulanya:

$$\text{Avalanche} = \frac{\text{jumlah bit cipherteks yang berubah}}{\text{jumlah seluruh bit cipherteks}} \times 100\%$$



Gambar 1.6 Contoh efek longsoran pada DES (Munir, 2026)

Entropi. Entropi Shannon mengukur derajat keacakan data. Untuk data pada tingkat byte:

$$H = -\sum_{i=0}^{255} p_i \log_2 p_i$$

Di sini p_i adalah peluang kemunculan byte bernilai i. Nilai maksimumnya adalah 8 bit/byte, yang tercapai jika semua nilai byte muncul dengan peluang yang sama. Cipherteks yang baik memiliki entropi mendekati 8.

Histogram. Histogram menampilkan frekuensi kemunculan setiap nilai byte (0 sampai 255). Plainteks umumnya memiliki histogram yang tidak merata, sedangkan cipherteks yang baik memiliki histogram yang mendekati distribusi seragam (*uniform*), sehingga tidak memberi petunjuk statistik tentang plainteksnya.

II. Perancangan *Block Cipher*

2.1 Gambaran Umum TUNG-128

TUNG-128 adalah cipher blok berbasis jaringan Feistel yang beroperasi pada blok 128 bit dengan kunci 128 bit. Gagasan utamanya adalah menggabungkan tiga jenis operasi dari kelompok aljabar yang berbeda di dalam satu fungsi putaran: substitusi kotak-S pada tingkat byte yang dibangun di atas GF(2^8) seperti pada AES, perkalian modular pada tingkat *word* 64 bit seperti pada RC6, dan rotasi bit yang jumlahnya berbeda pada tiap putaran. Operasi tingkat byte memberikan confusion, sedangkan operasi tingkat *word* menyebarkan pengaruh antarbyte yang tidak dapat dilakukan oleh kotak-S.

Nama TUNG-128 adalah singkatan dari nama lengkapnya, *Tung Block Cahur*, yang merupakan pelesetan dari frasa "Tung Tung Tung Sahur", yaitu bunyi kentungan untuk membangunkan sahur; angka 128 menyatakan ukuran blok dan kunci dalam bit. Nama ini tidak hanya menjadi label, tetapi juga tertanam di dalam rancangan: langkah difusi pada pembangkitan kunci putaran menggabungkan *state* dengan empat salinan dirinya yang dirotasi, yaitu tiga ketukan "Tung" (rotasi 11, 13, dan 14 byte) dan satu "Sahur" (rotasi 15 byte), seperti dijelaskan pada Subbab 2.4.6. Nama yang sama dipakai sebagai konstanta awal pembangkitan kunci putaran.

Jaringan Feistel dipilih agar fungsi putaran bebas dirancang serumit mungkin tanpa harus memiliki invers. Parameter TUNG-128 dirangkum pada Tabel 2.1.

Tabel 2.1 Parameter TUNG-128

Parameter	Nilai
Struktur	Jaringan Feistel
Ukuran blok	128 bit (16 byte), dibagi menjadi dua bagian 64 bit
Ukuran kunci utama	128 bit (16 byte)
Jumlah putaran	16
Kunci putaran	16 buah, masing-masing 128 bit
Kotak-S	8 × 8 bit, 256 entri
Satuan operasi	Byte (kotak-S, permutasi) dan *word* 64 bit (perkalian, rotasi)

Tabel 2.2 menunjukkan komponen yang memenuhi setiap prinsip dan teknik perancangan cipher blok

Tabel 2.2 Pemenuhan prinsip perancangan pada TUNG-128

Prinsip / teknik	Komponen TUNG-128

Confusion	Kotak-S dan perkalian modular di fungsi putaran; kotak-S di pembangkitan kunci putaran
Diffusion	Permutasi byte, perkalian modular, rotasi bit, dan pertukaran bagian kiri–kanan pada jaringan Feistel
Substitusi (*table lookup*)	Kotak-S 8 × 8 bit
Transposisi (permutasi)	Permutasi 8 byte di fungsi putaran; transposisi 4 × 4 di pembangkitan kunci putaran
Operasi tambahan	Perkalian modulo 2^{64} dan rotasi bit
Cipher berulang	16 putaran, masing-masing dengan kunci putaran berbeda

2.2 Proses Enkripsi

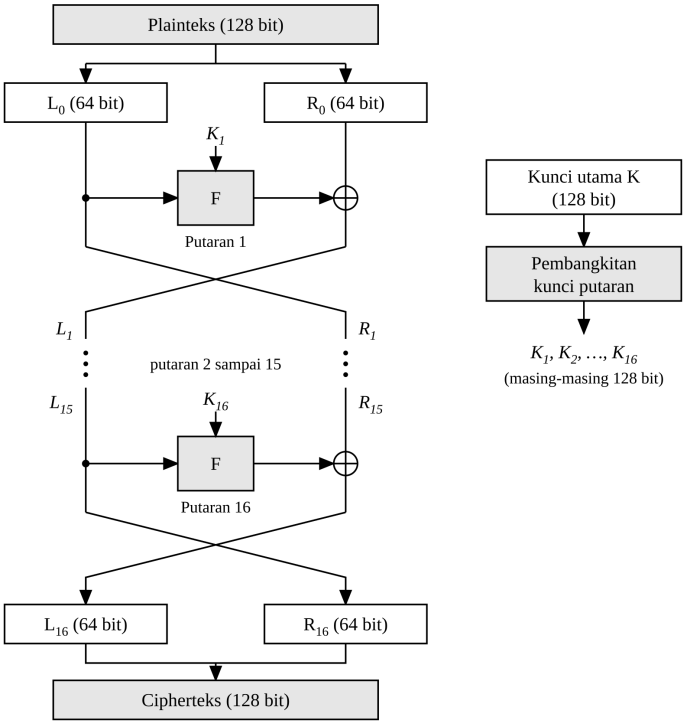
Enkripsi satu blok berlangsung dalam tiga tahap:

1. Enam belas kunci putaran K_1, K_2, …, K_{16} dibangkitkan dari kunci utama (Subbab 2.4.6).
2. Blok plainteks 128 bit dibagi menjadi bagian kiri L_0 dan bagian kanan R_0, masing-masing 64 bit.
3. Untuk $i = 1$ sampai 16, dilakukan satu putaran Feistel dengan fungsi putaran F (Subbab 2.4.1)

$$L_i = R_{i-1} \oplus F(L_{i-1}, K_i), \qquad R_i = L_{i-1}$$

Note: ini tidak terbalik, sengaja dibalik jaringan Feistel enkripsi yang berbeda dengan jaringan Feistel generik

Blok cipherteks adalah gabungan L_{16} dan R_{16}.



Gambar 2.1 Alur enkripsi TUNG-128

Dibandingkan dengan bentuk umum pada Subbab 1.5, peran kedua bagian dicerminkan: fungsi putaran dikenakan pada bagian kiri, dan hasilnya di-XOR-kan dengan bagian kanan. Pencerminan ini tidak mengubah sifat jaringan Feistel karena pada setiap putaran satu bagian tetap diteruskan apa adanya dan bagian lainnya diubah.

Jumlah putaran ditetapkan 16 dengan dua pertimbangan. Pertama, satu putaran Feistel hanya mengubah setengah blok, sehingga dibutuhkan beberapa putaran sebelum setiap bit cipherteks bergantung pada seluruh bit plainteks dan kunci. Kedua, 16 putaran memberi margin keamanan di atas jumlah minimum tersebut dengan waktu komputasi yang masih wajar, seperti pada DES.

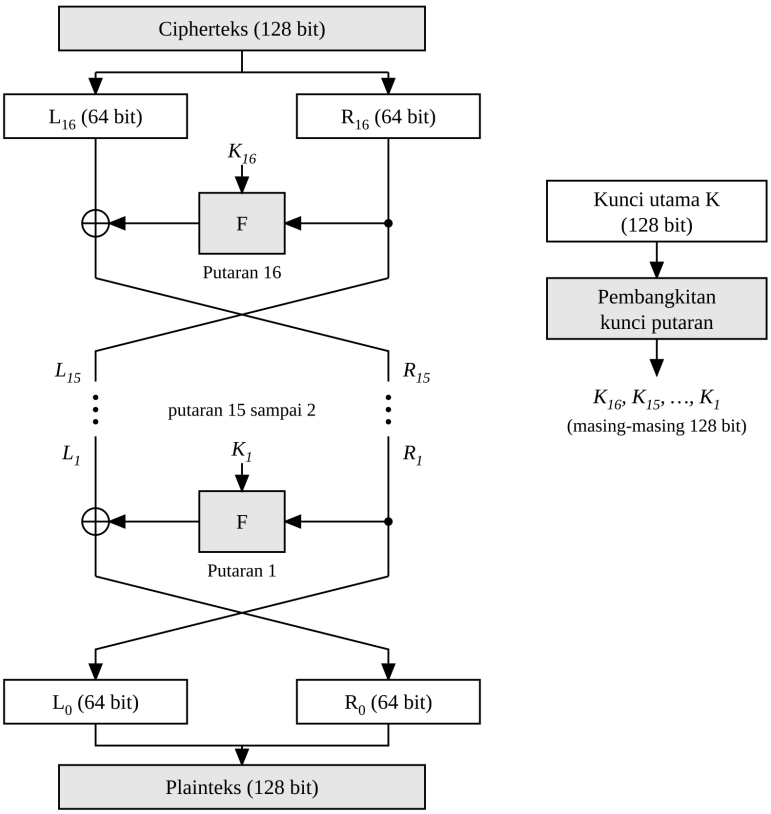
2.3 Proses Dekripsi

Dekripsi memakai struktur yang sama dengan enkripsi, tetapi putaran dijalankan mundur dari i = 16 sampai 1 dengan kunci putaran dalam urutan terbalik. Blok cipherteks dibagi menjadi L_{16} dan R_{16}, lalu pada setiap putaran berlaku:

$$L_{i-1} = R_i, \qquad R_{i-1} = L_i \oplus F(R_i, K_i)$$

Blok plainteks adalah gabungan L_0 dan R_0. Persamaan ini benar karena $R_i = L_{i-1}$, sehingga $F(R_i, K_i)$ sama dengan nilai yang dipakai saat enkripsi dan saling meniadakan lewat operasi XOR:

$$L_i \oplus F(R_i, K_i) = R_{i-1} \oplus F(L_{i-1}, K_i) \oplus F(L_{i-1}, K_i) = R_{i-1}$$



Gambar 2.2 Alur dekripsi TUNG-128

Akibat struktur ini, fungsi putaran yang dipakai saat dekripsi sama persis dengan saat enkripsi. Kotak-S invers dan permutasi invers tidak pernah dipakai dalam dekripsi blok; kotak-S invers tetap disajikan pada Subbab 2.4.2 untuk menunjukkan bahwa kotak-S bersifat bijektif.

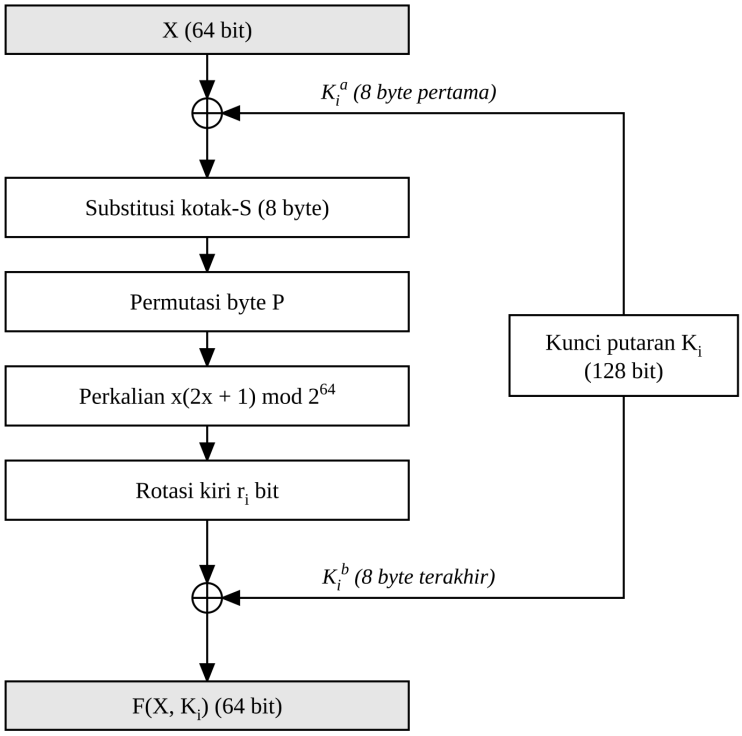
2.4 Komponen Algoritma

2.4.1 Fungsi Putaran

Fungsi putaran F menerima setengah blok X berukuran 64 bit dan kunci putaran K_i berukuran 128 bit, lalu menghasilkan keluaran 64 bit. Kunci putaran dibagi menjadi dua bagian 64 bit, yaitu K_i^{a} (8 byte pertama) dan K_i^{b} (8 byte terakhir). Fungsi F terdiri atas enam langkah berurutan:

$$
\begin{aligned}
A &= X \oplus K_i^a \\
B &= S(A) \\
C &= P(B) \\
D &= C\,(2C+1) \bmod 2^{64} \\
E &= D \lll r_i \\
F(X, K_i) &= E \oplus K_i^b
\end{aligned}
$$

Di sini S adalah substitusi kotak-S pada setiap byte, P adalah permutasi byte, dan r_i adalah jumlah rotasi pada putaran ke-i.



Gambar 2.3 Fungsi putaran TUNG-128

Urutan langkah dirancang dengan pertimbangan berikut:

- **XOR kunci di awal.** Masukan kotak-S bergantung pada kunci, sehingga hubungan antara kunci dan keluaran menjadi nirlanjar (non-linear).
- **Substitusi sebelum perkalian.** Kotak-S memberikan confusion pada setiap byte, tetapi satu byte masukan hanya memengaruhi satu byte keluaran.
- **Permutasi di antara keduanya.** Posisi byte menentukan bobotnya di dalam bilangan 64 bit, sehingga permutasi mengubah byte mana yang berada pada orde rendah sebelum perkalian.
- **Perkalian lalu rotasi.** Perkalian menyebarkan pengaruh antarbyte, tetapi hanya dari bit orde rendah ke orde tinggi; rotasi memindahkan bit orde tinggi yang paling tercampur ke posisi lain.
- **XOR kunci di akhir.** Keluaran fungsi ditutup dengan bagian kunci yang berbeda, sehingga seluruh 128 bit kunci putaran terpakai dan langkah rotasi tidak dapat langsung dibalik dari keluaran

2.4.2 Kotak-S

Kotak-S TUNG-128 memetakan 8 bit ke 8 bit dan dibangkitkan lewat perhitungan, bukan dipilih secara acak. Untuk setiap byte *a*, dilakukan dua langkah:

1. Hitung invers perkalian a^{-1} di dalam GF(2^8) dengan polinom tak-tereduksi $m(x) = x^8 + x^6 + x^5 + x^2 + 1$; nilai 00 dipetakan ke 00.
2. Kenakan transformasi affine pada hasilnya, dengan $y = a^{-1}$:

$$S(a) = y \oplus (y \lll 1) \oplus (y \lll 2) \oplus (y \lll 5) \oplus (y \lll 7) \oplus \mathtt{1F}, \qquad y = a^{-1}$$

Rotasi pada persamaan di atas adalah rotasi kiri di dalam satu byte, dan 1F adalah konstanta dalam heksadesimal. Hasilnya ditunjukkan pada Tabel 2.3; baris menyatakan 4 bit tinggi masukan dan kolom menyatakan 4 bit rendah. Tabel 2.4 adalah inversnya.

Tabel 2.3 Kotak-S TUNG-128 (heksadesimal)

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	1F	B8	0D	FC	16	C0	EE	92	5A	64	31	51	E7	ED	18	D4
1	BD	D8	63	7E	C9	9D	F9	30	A2	78	A7	36	9C	DA	FA	57
2	8F	27	3D	B9	E0	E2	6E	5C	74	59	9F	43	6C	86	88	B2
3	00	B0	AC	DC	82	BE	4A	BC	DE	7B	3C	8E	2C	C5	3B	61
4	96	CA	03	9A	0E	70	4C	28	21	48	20	42	66	11	7F	65
5	6B	7C	FD	85	9E	A1	F0	89	67	81	D3	71	15	07	08	01
6	90	4B	09	76	C6	7D	FE	D6	10	5F	CF	A9	B5	39	CE	35
7	FF	05	EC	02	4F	7A	D7	68	47	D2	B3	A3	CC	2B	E1	AB
8	DB	8C	34	4E	D0	C8	1C	72	56	F1	A8	25	77	F5	84	E5
9	C1	F2	B4	97	80	EB	B1	26	62	32	D9	2A	2F	A0	22	C3
A	E4	99	6F	52	AF	C7	93	24	DF	58	40	1B	29	E3	54	BF
B	23	87	50	5B	79	0B	E9	83	1A	C2	13	5E	94	0C	D1	4D
C	19	0F	F4	2D	D5	1D	6A	8A	F3	73	2E	5D	EF	CB	FB	3A
D	98	B7	3F	F8	B6	55	44	9B	8B	95	CD	53	F7	AA	0A	04
E	AE	EA	12	06	E6	DD	91	17	37	A5	AD	14	BA	BB	A4	1E
F	33	46	38	3E	49	69	41	E8	F6	6D	C4	8D	60	A6	45	75

Tabel 2.4 Kotak-S invers TUNG-128 (heksadesimal)

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	30	5F	73	42	DF	71	E3	5D	5E	62	DE	B5	BD	02	44	C1

1	68	4D	E2	BA	EB	5C	04	E7	0E	C0	B8	AB	86	C5	EF	00
2	4A	48	9E	B0	A7	8B	97	21	47	AC	9B	7D	3C	C3	CA	9C
3	17	0A	99	F0	82	6F	1B	E8	F2	6D	CF	3E	3A	22	F3	D2
4	AA	F6	4B	2B	D6	FE	F1	78	49	F4	36	61	46	BF	83	74
5	B2	0B	A3	DB	AE	D5	88	1F	A9	29	08	B3	27	CB	BB	69
6	FC	3F	98	12	09	4F	4C	58	77	F5	C6	50	2C	F9	26	A2
7	45	5B	87	C9	28	FF	63	8C	19	B4	75	39	51	65	13	4E
8	94	59	34	B7	8E	53	2D	B1	2E	57	C7	D8	81	FB	3B	20
9	60	E6	07	A6	BC	D9	40	93	D0	A1	43	D7	1C	15	54	2A
A	9D	55	18	7B	EE	E9	FD	1A	8A	6B	DD	7F	32	EA	E0	A4
B	31	96	2F	7A	92	6C	D4	D1	01	23	EC	ED	37	10	35	AF
C	05	90	B9	9F	FA	3D	64	A5	85	14	41	CD	7C	DA	6E	6A
D	84	BE	79	5A	0F	C4	67	76	11	9A	1D	80	33	E5	38	A8
E	24	7E	25	AD	A0	8F	E4	0C	F7	B6	E1	95	72	0D	06	CC
F	56	89	91	C8	C2	8D	F8	DC	D3	16	1E	CE	03	52	66	70

Pertimbangan rancangannya adalah sebagai berikut:

- **Invers di GF(2^8).** Operasi ini dipilih karena sifat nirlanjarnya (non-linear-nya) yang sudah teruji pada AES. Kotak-S yang dihasilkan memiliki nonlinearitas 112 dan *differential uniformity* 4, sama dengan kotak-S AES.
- **Polinom yang berbeda dari AES.** Polinom $x^8 + x^6 + x^5 + x^2 + 1$ menghasilkan medan dengan tabel perkalian yang berbeda, sehingga isi kotak-S berbeda dari kotak-S AES tanpa mengurangi kekuatannya.
- **Transformasi affine.** Transformasi ini menghilangkan bentuk aljabar sederhana dari operasi invers dan memastikan 00 tidak dipetakan ke 00.
- **Bijektif dan tanpa titik tetap.** Kotak-S memetakan 256 masukan ke 256 keluaran yang berbeda, dan tidak ada nilai x dengan $S(x) = x$.

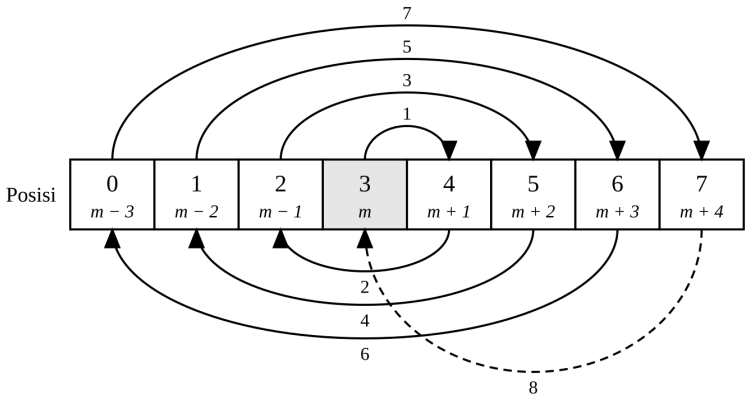
2.4.3 Permutasi Byte

Permutasi *P* mengacak posisi 8 byte keluaran kotak-S tanpa mengubah nilainya. Pemetaannya ditunjukkan pada Tabel 2.5, dengan posisi 0 sebagai byte paling kiri.

Tabel 2.5 Permutasi byte pada fungsi putaran

Posisi keluaran	0	1	2	3	4	5	6	7
Diambil dari posisi masukan	6	5	4	7	3	2	1	0

Permutasi ini disusun dari pola yang bergerak dari tengah ke luar secara berselang-seling. Dengan $m = 3$ sebagai posisi tengah, byte pada posisi m dipindahkan ke $m + 1$, byte pada $m + 1$ ke $m - 1$, byte pada $m - 1$ ke $m + 2$, dan seterusnya sampai posisi terluar; byte pada posisi terakhir kembali ke m. Hasilnya adalah satu siklus tunggal sepanjang 8, seperti pada Gambar 2.4.



Gambar 2.4 Pola permutasi byte; panah menunjukkan perpindahan byte dan angka pada panah menunjukkan urutan pola

Pertimbangan rancangannya adalah sebagai berikut:

- **Tidak ada byte yang diam.** Karena berupa satu siklus tunggal, setiap byte selalu berpindah posisi.
- **Perpindahan antara orde rendah dan tinggi.** Pola berselang-seling memindahkan byte bolak-balik antara separuh kiri dan separuh kanan, sehingga byte yang berbobot tinggi pada perkalian menjadi berbobot rendah, dan sebaliknya.

2.4.4 Perkalian Modular

Delapan byte hasil permutasi diperlakukan sebagai satu bilangan bulat 64 bit x, dengan byte pertama sebagai byte paling signifikan, lalu ditransformasikan dengan fungsi yang diambil dari RC6:

$$g(x) = x\,(2x + 1) \bmod 2^{64}$$

Operasi ini adalah operasi tambahan di luar substitusi dan permutasi. Pertimbangan rancangannya adalah sebagai berikut:

- **Difusi antarbyte.** Kotak-S dan permutasi bekerja per byte; perkalian adalah langkah pertama yang membuat satu byte masukan memengaruhi banyak byte keluaran.

- **Nirlanjar (non-linear) terhadap XOR.** Perkalian bilangan bulat menimbulkan *carry* antarbit, sehingga menambah confusion di luar kotak-S.
- **Bijektif.** Fungsi *g* memetakan setiap nilai 64 bit ke nilai yang berbeda, sehingga tidak ada informasi yang hilang di dalam fungsi putaran.

Kelemahan operasi ini adalah arah difusinya: bit orde rendah memengaruhi bit orde tinggi, tetapi tidak sebaliknya. Kelemahan ini ditutup oleh rotasi pada langkah berikutnya.

2.4.5 Rotasi Bit

Hasil perkalian dirotasi ke kiri secara sirkuler sebagai satu bilangan 64 bit. Jumlah rotasi r_i berbeda untuk setiap putaran, yaitu 16 bilangan prima pertama, seperti pada Tabel 2.6.

Tabel 2.6 Jumlah rotasi bit pada setiap putaran

Putaran	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Rotasi (bit)	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53

Pertimbangan rancangannya adalah sebagai berikut:

- **Menyebarkan bit orde tinggi.** Bit orde tinggi hasil perkalian adalah bit yang paling tercampur; rotasi memindahkannya ke posisi lain sehingga pada putaran berikutnya ikut memengaruhi bit orde rendah.
- **Tidak sejajar dengan batas byte.** Tidak ada jumlah rotasi yang merupakan kelipatan 8, sehingga bit-bit dari satu byte selalu terpecah ke dua byte yang bersebelahan sebelum masuk ke kotak-S pada putaran berikutnya.
- **Berbeda di setiap putaran.** Jumlah rotasi yang berubah-ubah membuat setiap putaran menjadi transformasi yang berbeda, sehingga tidak ada pola pergeseran yang berulang antarputaran.

2.4.6 Pembangkitan Kunci Putaran

Pembangkitan kunci putaran mengubah kunci utama K (16 byte) menjadi 16 kunci putaran berukuran 16 byte. Prosesnya memakai sebuah *state* 16 byte yang diperbarui pada setiap putaran, dan nilai *state* itu sendiri menjadi kunci putarannya.

Inisialisasi. *State* awal diperoleh dengan meng-XOR-kan kunci utama dengan konstanta tetap W, dan sebuah salinan kunci utama yang sudah ditransposisikan disimpan sebagai M:

$$T_0 = K \oplus W, \qquad M = \mathrm{Tr}(K)$$

Konstanta W adalah 16 byte kode ASCII dari teks **`TUNG-128-KERNEL!`**.

Putaran. Untuk i = 1 sampai 16:

$$T_i = \mathrm{Df}\big(\mathrm{Tr}(S(T_{i-1}))\big) \oplus RC_i \oplus \big(M \lll_{\text{byte}} 5(i-1) \bmod 16\big), \qquad K_i = T_i$$

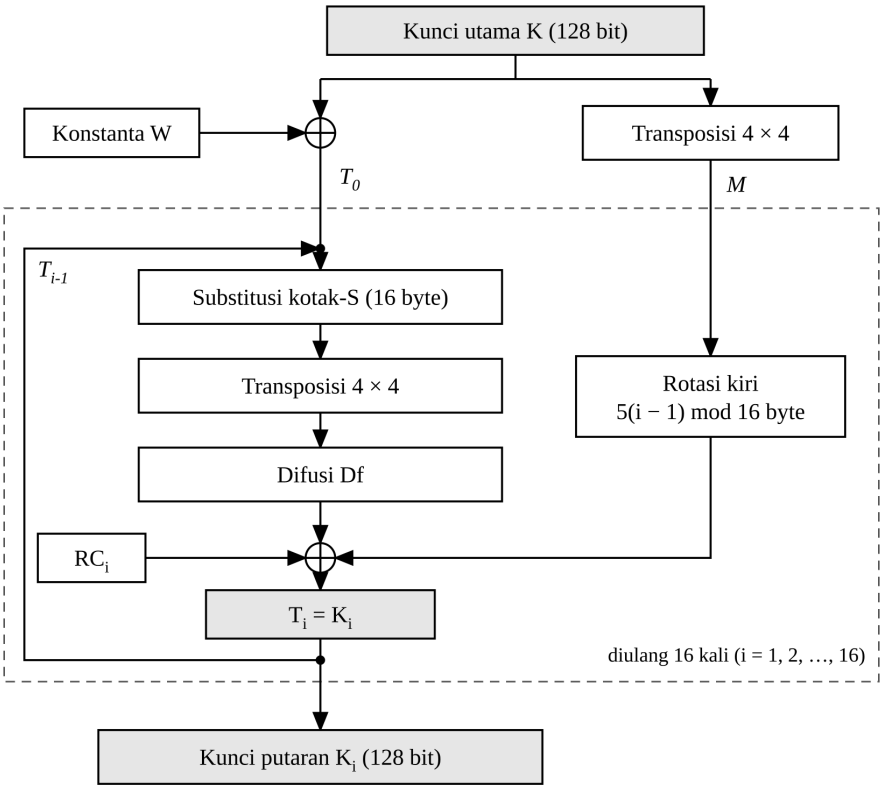
Operasi yang dipakai adalah sebagai berikut:

- **Substitusi *S*.** Setiap byte *state* disubstitusi dengan kotak-S yang sama dengan fungsi putaran.
- **Transposisi Tr.** Enam belas byte disusun sebagai matriks 4 × 4, lalu matriks tersebut ditransposisikan (baris menjadi kolom).
- **Difusi Df.** Setiap byte keluaran adalah XOR dari lima byte masukan, dengan indeks dihitung dalam modulo 16:

$$\mathrm{Df}(s)_j = s_j \oplus s_{j+11} \oplus s_{j+13} \oplus s_{j+14} \oplus s_{j+15}$$

Konstanta putaran RC_i. Enam belas byte yang berbeda untuk setiap putaran. Konstanta ini dibangkitkan secara deterministik dari nomor putaran dengan perkalian berulang di GF(2^8) yang hasilnya dilewatkan ke kotak-S.

Pengikatan kunci utama. Salinan kunci M dirotasi sejauh $5(i - 1) \bmod 16$ byte, lalu di-XOR-kan ke *state*.



Gambar 2.5 Pembangkitan kunci putaran TUNG-128

Pertimbangan rancangannya adalah sebagai berikut:

- **XOR dengan konstanta di awal.** Kunci yang lemah seperti kunci nol semua tidak menghasilkan *state* awal yang nol.

- **Substitusi.** Kunci putaran menjadi fungsi nirlanjar dari kunci utama, sehingga mengetahui satu kunci putaran tidak langsung memberikan kunci lainnya lewat persamaan linear.
- **Transposisi dan difusi.** Setiap byte kunci utama memengaruhi banyak byte kunci putaran; setelah beberapa putaran, setiap byte kunci putaran bergantung pada seluruh byte kunci utama.
- **Konstanta putaran.** Setiap putaran pembangkitan menjadi berbeda, sehingga kunci putaran tidak berulang dan cipher tidak memiliki kesamaan antarputaran yang dapat dieksploitasi.
- **Pengikatan kunci utama.** Kunci utama dimasukkan kembali di setiap putaran dengan posisi byte yang berbeda, sehingga setiap kunci putaran bergantung langsung pada kunci utama, bukan hanya pada kunci putaran sebelumnya.
- **Kunci putaran 128 bit.** Ukuran ini dua kali setengah blok, sehingga fungsi putaran dapat memakai dua bagian kunci yang berbeda di awal dan di akhir.

III. Implementasi dan Simulasi

3.1. Deskripsi Program

TUNG-128 adalah program desktop berbasis Python untuk mengenkripsi dan mendekripsi berkas serta menganalisis hasil enkripsi. Program menyediakan lima mode operasi, yaitu ECB, CBC, CFB, OFB, dan CTR. Selain mengolah berkas, program menghitung entropi byte, menampilkan histogram plaintext dan ciphertext, serta mengukur avalanche effect dengan membandingkan ciphertext sebelum dan sesudah satu bit plaintext atau kunci diubah.

Hasil enkripsi disimpan sebagai berkas dengan format envelope yang memuat informasi mode, IV atau counter jika diperlukan, ciphertext, dan tag integritas. Saat dekripsi, program memeriksa tag terlebih dahulu. Jika pemeriksaan berhasil, ciphertext didekripsi dan hasilnya dapat disimpan sebagai berkas plaintext.

3.2. Deskripsi Antarmuka

Antarmuka program terdiri atas bagian pemilihan berkas, pengaturan kunci dan mode, tombol operasi, area hasil analisis, serta grafik histogram.

- **Pemilihan berkas**

  Pengguna memilih berkas yang akan dienkripsi, didekripsi, atau dianalisis melalui tombol **Choose file…**. Berkas masukan dapat berupa teks maupun data biner.

- **Input kunci utama**

  Kunci dimasukkan dalam bentuk heksadesimal. Implementasi saat ini menerima kunci sepanjang 16 byte, atau 32 digit heksadesimal.

- **Pemilihan mode operasi**

  Pengguna memilih salah satu mode: ECB, CBC, CFB, OFB, atau CTR. Pemilihan mode ini digunakan saat enkripsi dan analisis. Saat dekripsi, mode yang dipilih harus sesuai dengan mode yang tersimpan pada berkas terenkripsi.

- **Input IV atau counter**

  Untuk CBC, CFB, OFB, dan CTR, pengguna dapat memasukkan IV atau nilai counter awal sepanjang 16 byte dalam bentuk heksadesimal. Tombol **Generate** membuat nilai baru secara otomatis. ECB tidak menggunakan IV.

- **Tombol operasi**

  Encrypt… mengenkripsi berkas dan meminta lokasi penyimpanan hasil, **Decrypt…** memeriksa integritas, mendekripsi berkas, lalu meminta lokasi penyimpanan plaintext, dan **Analyze + avalanche** menghitung statistik plaintext dan ciphertext untuk kelima mode, kemudian menampilkan hasil perbandingan.

- **Area hasil analisis**

  Area teks menampilkan entropi plaintext, entropi ciphertext, jumlah byte ciphertext, serta persentase perubahan bit ciphertext untuk pengujian *plaintext flip* dan *key flip* pada setiap mode.

- **Grafik Histogram**

Grafik pertama menunjukkan distribusi frekuensi byte plaintext. Grafik kedua membandingkan distribusi byte ciphertext dari kelima mode. Nilai entropi ditampilkan bersama histogram plaintext.

3.3. Hasil Pengujian Enkripsi dan Dekripsi

Berikut merupakan hasil pengujian enkripsi dan dekripsi dengan salah satu berkas uji, yaitu *plaintext.txt*. Hasil pengujian dapat didapatkan secara manual dan dilakukan satu per satu melalui program utama, tetapi terdapat sebuah script pengujian untuk langsung menguji semua pengujian dengan setiap mode enkripsi (CBC, ECB, dsb.)

- **Uji *Avalanche Effect***: Menghitung persentase perubahan bit pada *ciphertext* ketika 1 bit *plaintext* atau 1 bit kunci diubah.

Avalanche dan entropi

Avalanche menguji tiap bit pada 32 byte plaintext. Entropi maksimum byte adalah 8 bit.

Mode	Byte plaintext	Byte ciphertext	Entropi plaintext	Entropi ciphertext	Avalanche key rata-rata (%)	Avalanche plaintext rata-rata (%)
ECB	5844	5856	4.6372	7.9694	50.1058	16.6168
CBC	5844	5856	4.6372	7.9671	50.3052	41.5914
CFB	5844	5856	4.6372	7.9671	49.8271	25.3276
OFB	5844	5856	4.6372	7.9664	49.9084	0.2604
CTR	5844	5856	4.6372	7.9692	50.1444	0.2604

Nilai Avalanche key pada seluruh mode berada di sekitar 50%, dengan nilai tertinggi pada CBC sebesar 50,3052% dan terendah pada CFB sebesar 49,8271%. Hasil ini menunjukkan bahwa perubahan pada kunci menyebabkan sekitar setengah bit ciphertext berubah, sesuai dengan karakteristik efek avalanche yang diharapkan pada algoritma kriptografi yang baik. Sedangkan Nilai Avalanche plaintext tertinggi diperoleh oleh CBC sebesar 41,5914%, diikuti CFB sebesar 25,3276% dan ECB sebesar 16,6168%. Sementara itu, OFB dan CTR menghasilkan nilai sebesar 0,2604%. Nilai yang sangat rendah pada OFB dan CTR dapat terjadi karena kedua mode tersebut menggunakan keystream yang tidak bergantung pada plaintext. Dengan demikian, perubahan plaintext akan memengaruhi bit ciphertext pada posisi yang bersesuaian, bukan menyebar ke seluruh ciphertext.

- **Analisis Entropi**: Menghitung derajat keacakan byte *ciphertext*. Untuk *ciphertext* tingkat byte, nilai maksimum entropinya adalah 8 bit/byte.

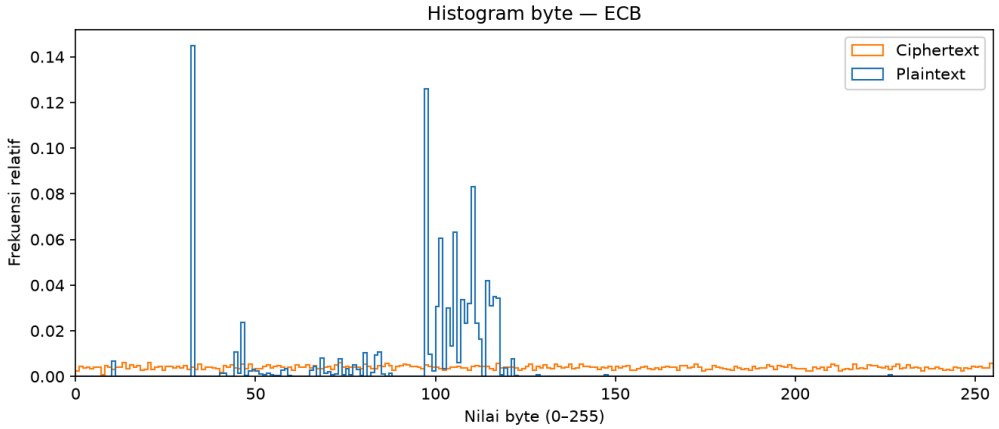
Avalanche dan entropi

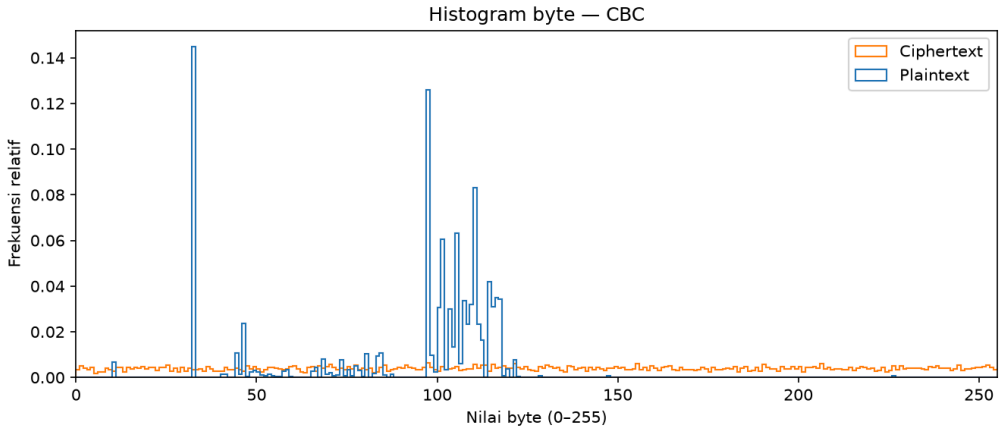
Avalanche menguji tiap bit pada 32 byte plaintext. Entropi maksimum byte adalah 8 bit.

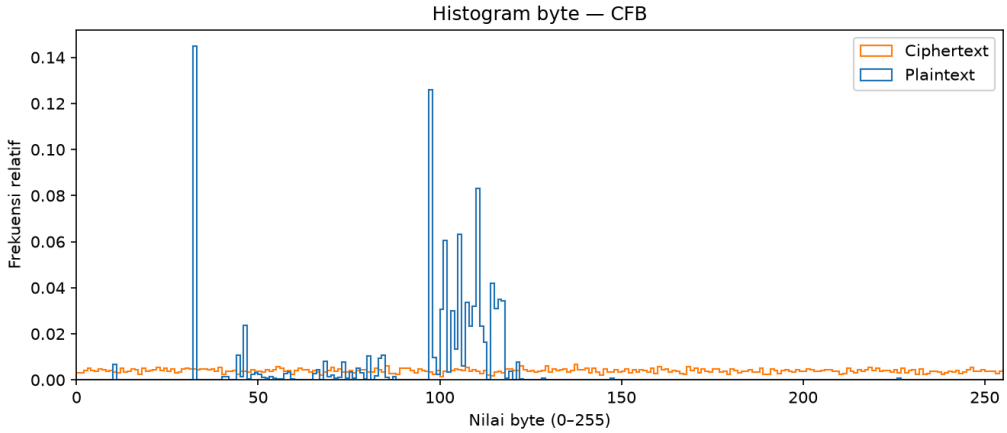
Mode	Byte plaintext	Byte ciphertext	Entropi plaintext	Entropi ciphertext	Avalanche key rata-rata (%)	Avalanche plaintext rata-rata (%)
ECB	5844	5856	4.6372	7.9694	50.1058	16.6168
CBC	5844	5856	4.6372	7.9671	50.3052	41.5914
CFB	5844	5856	4.6372	7.9671	49.8271	25.3276
OFB	5844	5856	4.6372	7.9664	49.9084	0.2604
CTR	5844	5856	4.6372	7.9692	50.1444	0.2604

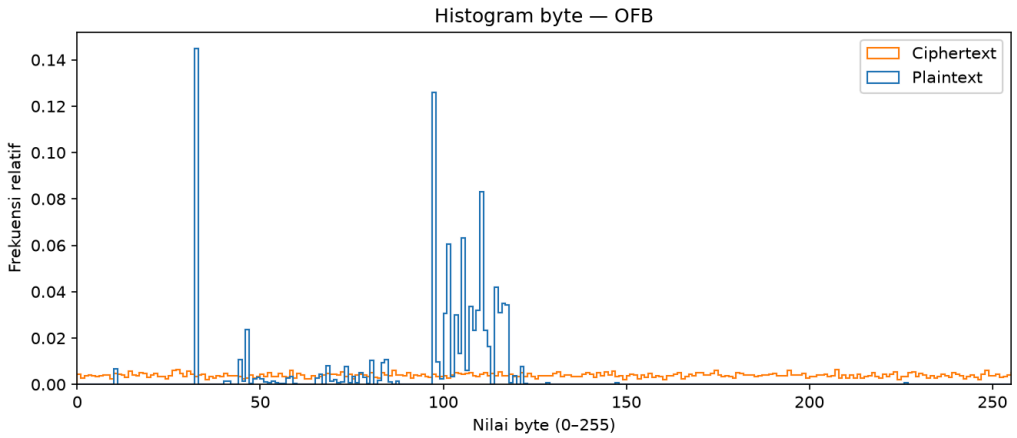
Berdasarkan pengujian, enkripsi berhasil membuat nilai entropy meningkat sangat tinggi, dari plaintext yang nilai entropy-nya rendah sekarang hampir mendekati 8 yaitu sekitar ~7,9.

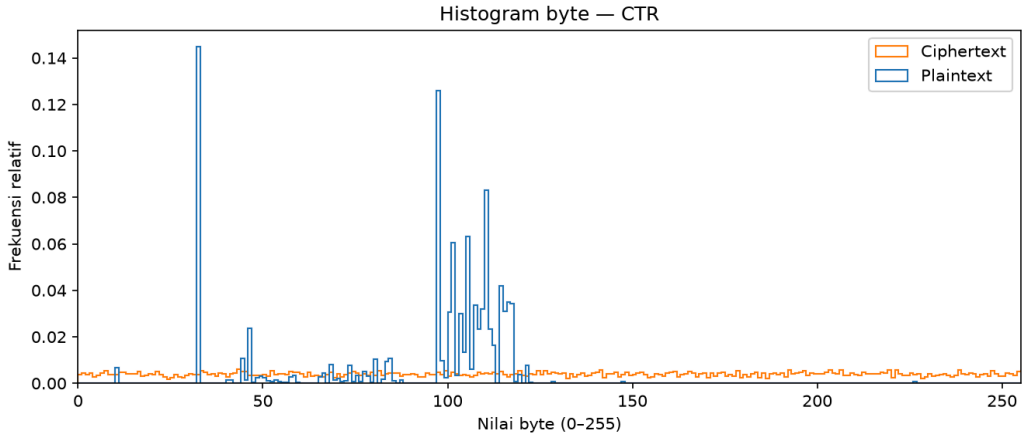
- **Analisis Histogram**: Membandingkan grafik distribusi frekuensi kemunculan byte antara *plaintext* dan *ciphertext* untuk melihat apakah sebaran *ciphertext* mendekati distribusi seragam (*uniform*) atau tidak.











Dari semua pengujian, terlihat bahwa semua metode enkripsi bisa membuat frekuensi karakter menjadi tersebar sehingga mengurangi kemungkinan *cryptanalyst* dari frekuensi kemunculan karakter. (atau dalam ini adalah byte)

- **Pengujian Tambahan 1 (enkripsi lalu dekripsi kembali)**

Round-trip berkas

Sampel	Mode	Byte	Hasil
plaintext.txt	ECB	5844	Sesuai/Berhasil
plaintext.txt	CBC	5844	Sesuai/Berhasil
plaintext.txt	CFB	5844	Sesuai/Berhasil
plaintext.txt	OFB	5844	Sesuai/Berhasil
plaintext.txt	CTR	5844	Sesuai/Berhasil
binary.bin	ECB	512	Sesuai/Berhasil
binary.bin	CBC	512	Sesuai/Berhasil
binary.bin	CFB	512	Sesuai/Berhasil
binary.bin	OFB	512	Sesuai/Berhasil
binary.bin	CTR	512	Sesuai/Berhasil

Dari hasil pengujian, algoritma sudah benar karena plainteks yang dienkripsi berhasil dikembalikan ke plainteks yang sama setelah didekripsi.

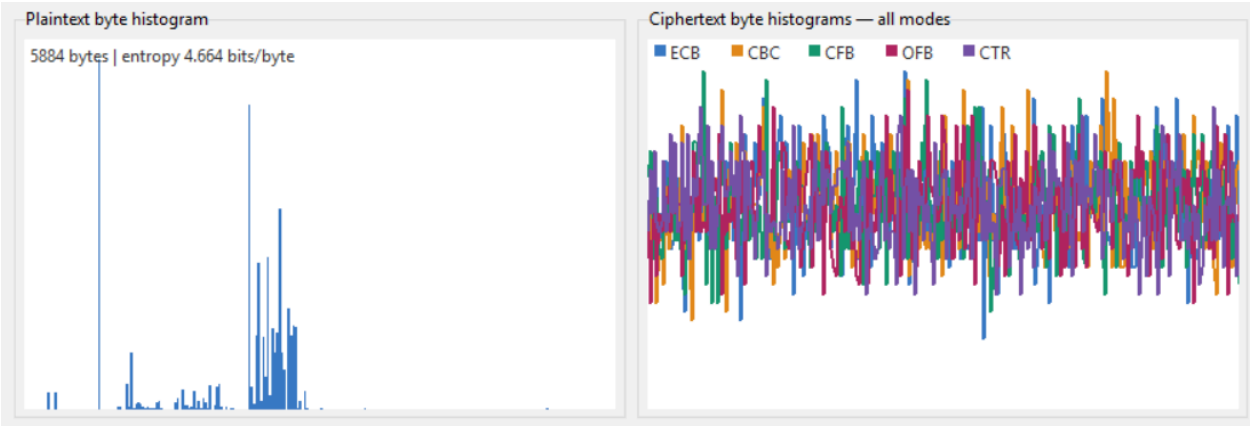
IV. Analisis Keamanan

Pengujian keamanan dilakukan terhadap sebuah berkas teks menggunakan lima mode operasi, yaitu ECB, CBC, CFB, OFB, dan CTR. Setiap mode menggunakan plaintext, kunci, dan IV atau counter yang sama agar hasilnya dapat dibandingkan secara terkontrol. Ukuran ciphertext yang dihasilkan pada setiap mode adalah 5.888 byte. Analisis mencakup pengukuran entropi byte, pengamatan histogram plaintext dan ciphertext, serta pengujian *avalanche effect* dengan membalik satu bit pada plaintext dan kunci. Hasil yang diperoleh menggambarkan perilaku algoritma pada berkas yang diuji, sehingga kesimpulan dari pengujian ini perlu dibatasi pada sampel tersebut.

Tabel 4.1 Analisis Keamanan

Mode	Plain H	Cipher H	Cipher bytes	PT Flip (%)	Key Flip (%)
ECB	4.6644	7.9618	5888	0.13	50.26
CBC	4.6644	7.9661	5888	50.06	50.00
CFB	4.6644	7.9672	5888	49.70	50.35
OFB	4.6644	7.9669	5888	0.00	49.78
CTR	4.6644	7.9702	5888	0.00	50.12

Entropi plaintext tercatat sebesar 4,6644 bit/byte. Nilai ini menunjukkan bahwa byte pada berkas teks tidak tersebar merata di antara seluruh kemungkinan nilai 0 sampai 255. Hal tersebut wajar karena teks biasanya mengandung pola, seperti karakter yang sering muncul, spasi, tanda baca, dan susunan kata tertentu. Setelah dienkripsi, entropi ciphertext meningkat menjadi 7,9618 pada ECB, 7,9661 pada CBC, 7,9672 pada CFB, 7,9669 pada OFB, dan 7,9702 bit/byte pada CTR. Nilai maksimum entropi byte adalah 8 bit/byte, sehingga semua hasil ciphertext pada pengujian ini berada dekat dengan nilai maksimum. Ini menunjukkan bahwa frekuensi kemunculan byte ciphertext dalam sampel lebih merata dibandingkan frekuensi byte plaintext. Pengukuran entropi mengikuti konsep entropi Shannon, yang mengukur ketidakpastian distribusi simbol. Namun, entropi tinggi hanya menggambarkan distribusi byte dan tidak dengan sendirinya membuktikan bahwa suatu algoritma aman.



Gambar 4.1 Histogram Plaintext dan Ciphertext

Hasil entropi tersebut dapat dibandingkan dengan histogram byte. Histogram plaintext umumnya memperlihatkan beberapa nilai byte yang lebih sering muncul daripada nilai lainnya karena karakter teks tidak memiliki frekuensi yang sama. Sebaliknya, jika histogram ciphertext yang dihasilkan program terlihat lebih rata, hal itu selaras dengan nilai entropi ciphertext yang mendekati 8 bit/byte. Dengan kata lain, pola frekuensi karakter yang tampak pada plaintext menjadi lebih sulit diamati dari distribusi byte ciphertext pada berkas yang diuji. Histogram tetap perlu ditafsirkan sebagai ringkasan statistik: histogram tidak memperhitungkan urutan byte, dan kemiripan histogram dengan distribusi seragam tidak cukup untuk membuktikan ketahanan terhadap serangan kriptografi.

Pengujian *avalanche effect* dilakukan dengan membandingkan ciphertext awal dengan ciphertext baru setelah satu bit input diubah. Persentase perubahan dihitung dari jumlah bit ciphertext yang berbeda dibagi jumlah seluruh bit ciphertext, kemudian dikalikan 100%. Untuk perubahan satu bit kunci, hasilnya adalah 50,26% pada ECB, 50,00% pada CBC, 50,35% pada CFB, 49,78% pada OFB, dan 50,12% pada CTR. Nilai-nilai tersebut berada dekat dengan 50%. Pada sampel ini, hasil tersebut menunjukkan bahwa perubahan satu bit kunci diikuti perubahan pada sekitar separuh bit ciphertext.

Untuk perubahan satu bit plaintext, hasil pada CBC adalah 50,06% dan pada CFB sebesar 49,70%. Kedua hasil ini mendekati 50%, yang berarti perubahan plaintext menyebar ke banyak bit ciphertext pada pengujian tersebut. Pada CBC, setiap blok plaintext dipengaruhi oleh ciphertext blok sebelumnya. Karena itu, perubahan pada blok awal dapat memengaruhi hasil enkripsi pada blok-blok selanjutnya. Pada CFB, ciphertext yang dihasilkan menjadi masukan umpan balik untuk proses berikutnya, sehingga perubahan dapat menyebar sepanjang proses enkripsi. Hasil tersebut perlu dibaca sebagai perilaku gabungan cipher dan mode operasi, bukan semata-mata ukuran difusi internal cipher blok.

Pada ECB, perubahan plaintext tercatat sebesar 0,13%. Angka ini rendah karena ECB memproses setiap blok secara terpisah: perubahan pada satu blok plaintext memengaruhi ciphertext blok tersebut, sementara blok lain tidak bergantung pada blok yang berubah. Persentase dalam program dihitung terhadap keseluruhan ciphertext berukuran 5.888 byte, bukan hanya terhadap blok yang terdampak. Akibatnya, perubahan yang cukup besar pada satu blok

dapat tampak kecil setelah dibagi dengan seluruh bit ciphertext. Oleh sebab itu, angka 0,13% pada ECB tidak berarti tidak ada perubahan; angka tersebut menunjukkan bahwa perubahan hanya mencakup bagian kecil dari keseluruhan berkas.

Pada OFB dan CTR, perubahan plaintext masing-masing tercatat sebesar 0,00%. Hasil ini berasal dari karakteristik mode yang menggunakan aliran kunci untuk melakukan operasi XOR dengan plaintext. Jika satu bit plaintext dibalik, umumnya bit ciphertext yang bersesuaian juga berubah, sedangkan bit lain tetap. Dengan total 5.888 byte atau 47.104 bit ciphertext, perubahan satu bit hanya setara dengan sekitar 0,0021% dari seluruh ciphertext. Nilai sekecil itu dibulatkan menjadi 0,00% pada tampilan program. Karena itu, hasil OFB dan CTR tidak dapat ditafsirkan dengan cara yang sama seperti hasil CBC dan CFB; angka tersebut sesuai dengan perubahan bit yang bersifat lokal pada mode tersebut.

Berdasarkan pengujian ini, ciphertext pada semua mode memiliki entropi yang mendekati nilai maksimum, dan perubahan satu bit kunci menghasilkan perubahan ciphertext sekitar 50%. Perubahan satu bit plaintext juga menghasilkan perubahan sekitar 50% pada CBC dan CFB, sedangkan ECB, OFB, dan CTR menunjukkan nilai rendah karena cara masing-masing mode memproses perubahan plaintext dan karena perhitungan dilakukan terhadap keseluruhan berkas. Temuan ini mendeskripsikan hasil untuk satu berkas teks dan satu kombinasi kunci serta IV/counter. Pengujian dengan berkas lain, termasuk teks dengan pola berbeda dan data biner, serta pengulangan dengan beberapa kunci dan IV/counter dapat memberikan gambaran yang lebih luas. Meskipun hasil entropi, histogram, dan avalanche dapat digunakan untuk mengevaluasi karakteristik statistik ciphertext, pengujian tersebut bukan bukti menyeluruh bahwa algoritma bebas dari kelemahan atau aman terhadap semua jenis serangan.

V. Kesimpulan dan Saran Pengembangan

5.1. Kesimpulan

Berdasarkan pengujian pada berkas teks berukuran 5.888 byte, entropi plaintext sebesar 4,6644 bit/byte meningkat menjadi 7,9618–7,9702 bit/byte setelah enkripsi pada kelima mode. Nilai tersebut menunjukkan bahwa distribusi byte ciphertext pada sampel ini lebih merata daripada plaintext. Pengujian *avalanche effect* menunjukkan bahwa pembalikan satu bit kunci mengubah sekitar 49,78–50,35% bit ciphertext. Untuk pembalikan satu bit plaintext, CBC dan CFB menghasilkan perubahan mendekati 50%, sedangkan nilai ECB, OFB, dan CTR lebih rendah karena cara mode tersebut menyebarkan perubahan dan karena persentase dihitung terhadap seluruh ciphertext. Hasil ini menggambarkan karakteristik statistik program pada sampel yang diuji, tetapi belum cukup untuk menyatakan bahwa algoritma aman terhadap semua jenis serangan.

5.2. Saran Pengembangan

Pengujian selanjutnya sebaiknya menggunakan lebih banyak jenis dan ukuran berkas, termasuk teks dengan pola berbeda serta data biner, dan diulang dengan beberapa kunci serta IV/counter. Uji *avalanche effect* dapat diperluas dengan membalik bit pada berbagai posisi plaintext dan kunci, mengulang setiap pengujian, serta mencatat rata-rata dan variasi hasil. Untuk membedakan difusi cipher dari pengaruh mode operasi, pengujian plaintext satu blok dapat dilakukan terpisah dari pengujian seluruh berkas. Program juga dapat dikembangkan dengan menampilkan histogram secara lebih jelas, menyimpan hasil analisis agar mudah dibandingkan, dan menguji bahwa dekripsi berhasil serta perubahan pada ciphertext atau tag integritas terdeteksi. Hasil pengujian statistik tersebut tetap perlu dilengkapi kajian kelemahan algoritma sebelum menarik kesimpulan keamanan yang lebih luas.

VI. Daftar Pustaka

Munir, R. (2026). 08 - Block Cipher [Slide presentasi]. Bahan Kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf

Munir, R. (2026). 09 - Review Beberapa Block Cipher (Bagian 1: DES dan Triple DES) [Slide presentasi]. Bahan Kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/09-Beberapa-block-cipher-bagian1-2026.pdf

Munir, R. (2026). 10 - Review Beberapa Block Cipher (Bagian 2: Advanced Encryption Standard - AES) [Slide presentasi]. Bahan Kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/10-Beberapa-block-cipher-bagian2-2026.pdf

Munir, R. (2026). 11 - Review Beberapa Block Cipher (Bagian 3: GOST, RC5, RC6, Serpent, dll) [Slide presentasi]. Bahan Kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/11-Beberapa-block-cipher-bagian3-2026.pdf

National Institute of Standards and Technology. (2001). Recommendation for block cipher modes of operation: Methods and techniques (NIST Special Publication 800-38A). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-38A

Stallings, W. (2020). Cryptography and network security: Principles and practice (8th ed.). Pearson.

Webster, A. F., & Tavares, S. E. (1986). On the design of S-boxes. Dalam H. C. Williams (Ed.), Advances in Cryptology - CRYPTO ’85 Proceedings (hlm. 523–534). Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-39799-X_41

Lampiran

1. Pranala Repositori Source Code: https://github.com/Fajar2k5/tugas-2-if4020-kripto

2. Tabel pembagian tugas masing-masing anggota

NIM	Nama	Tugas
13523017	Orvin Andika Ikhsan Abhista	• GUI • Perhitungan entropi dan histogram • Integrasi tools analisis • Build executable • Analisis entropi dan histogram semua mode • API Documentation (bonus)
13523027	Fajar Kurniawan	• S-box dan inversnya • Key schedule • Unit test S-box dan round key • Uji avalanche (kunci dan plaintext)
13523034	Rafizan Muhammad Syawalazmi	• Mode ECB, CBC, CFB, OFB, CTR • Padding dan unpadding • Verifikasi integritas • Penanganan IV / counter • Uji enkripsi–dekripsi file teks dan biner • Uji file kecil, sedang, besar
13523100	Aryo Wisanggeni	• Permutasi dan inversnya • Operasi tambahan cycling dan multiplier • Round function dan iterasi N ronde • Fungsi encrypt dan decrypt • Unit test fungsi blok